

RSTGen: Imbuing Fine-Grained Interpretable Control into Long-Form Text Generators

Rilwan A. Adewoyin^{1,4}, Ritabrata Dutta², Yulan He^{1,3}

¹Department of Computer Science, University of Warwick, UK

²Department of Statistics, University of Warwick, UK

³The Alan Turing Institute, UK

⁴Department of Computer Science and Engineering,

Southern University of Science and Technology, China

{rilwan.adewoyin, ritabrata.dutta, yulan.he}@warwick.ac.uk

Abstract

In this paper, we study the task of improving the cohesion and coherence of long-form text generated by language models. To this end, we propose RSTGen, a framework that utilises Rhetorical Structure Theory (RST), a classical language theory, to control the discourse structure, semantics and topics of generated text. Firstly, we demonstrate our model’s ability to control structural discourse and semantic features of generated text in open generation evaluation. Then we experiment on the two challenging long-form text tasks of argument generation and story generation. Evaluation using automated metrics and a metric with high correlation to human evaluation, shows that our model performs competitively against existing models, while offering significantly more controls over generated text than alternative methods.

1 Introduction

Controllable text generation has attracted much attention in recent years. Generating coherent and cohesive long-form text with controllable attributes is particularly challenging due to the relatively complex syntactic and semantic structures involved compared to short-text generation. Long-form text generation can find applications in automatic argumentation, motivational speech, and opinionated writing, to name a few.

A popular approach to controllable text generation is to design prompts, which control high-level linguistic features (i.e., text style and sentiment) by prefixing simple context to the input of a language model. These non-granular methods are often too coarse to allow different parts of the text to have diverse or contrasting features. Furthermore, they tend to focus on a single linguistic feature of text, meaning that extra frameworks such as PlugandPlay (Dathathri et al., 2019) are required to control multiple linguistic features.

To achieve improved content control, researchers have coupled a content planning module with a text generation module (Hua and Wang, 2020; Hua et al., 2021), in which the content planning module firstly generates a set of keyphrases and their corresponding positions in sentences and the generation module fills in the gaps. To control syntactic structures of text, recent works have proposed a neuro-symbolic approach leveraging Dependency Structure Theory (DST) (Shen et al., 2019; Du et al., 2020). Enforcing specific structures over the generated text has been proven to be useful in increasing coherence and cohesion of short-form text. However, a relatively large amount of DST information is required to encode comparatively short texts. Thus, they are unsuitable for use in long-form text generation tasks.

Similar to DST, Rhetorical Structure Theory (RST) (Mann and Thompson, 1988) is a classical tree-based interpretation of natural language. Whereas DST is most useful for intra-sentence interpretation, RST is most useful for inter-sentence interpretation of language. We propose a neurosymbolic framework that can imbue an RST understanding of text to existing language models. More specifically, our framework (1) allows more fine-grained control of syntax, semantics and text structure; (2) utilises a well-defined rhetorical structure of language, thus offering better interpretability; (3) can be directly integrated into existing pre-trained language models such as GPT-2 (Radford et al., 2019) and BART (Lewis et al., 2019).

We evaluate our proposed framework on two tasks: argument generation and story generation. We show that our proposed framework improves upon existing content control approaches on automatic evaluation metrics including BLEU (Papineni et al., 2002), METEOR (Denkowski and Lavie, 2014) and generates better text in terms of grammar and coherence measure. We further demonstrate our model’s ability to generate text

with control over syntactic, semantic and discourse features. Our main contributions can be summarised below:

- We propose a novel framework to imbue the RST information into pre-trained language models.
- We develop the first neurosymbolic framework that provides interpretable fine-control over syntax, semantics, discourse structure, topical keywords and keyword positions.
- We demonstrate the superiority of our proposed framework over existing planning and control methods for two long-form text generation tasks.

2 Related Work

We discuss several lines of research in controllable text generation.

Prompts For prompt-based control, a *context* is prepended to a language model input. The prepended item is related to the type of desired output. This method has been used to manipulate the syntax (Dušek and Jurčiček, 2016; Goyal and Durrett, 2020) and semantics (Wen et al., 2015; Chen et al., 2019) of the output. Alternatively, the prepended item can provide semantic control in order to cover a given topic (Wang et al., 2019a), mention specified entities (Fan et al., 2019), display a certain attribute (Hu et al., 2017; Balakrishnan et al., 2019), or even exhibit a style of text (Keskar et al., 2019). These methods suffer from the inability to exert fine-control, that is, a change in any one of the input prompts will change the whole generated text. Furthermore, all these works utilise non-expansive features for their prompts, which prevents them from making iterative improvements to existing generated text.

Content Planning Despite the impressive progress made in many generation tasks, earlier neural models are known to produce low-quality content (Wiseman et al., 2017), often with low relevance, and poor discourse structure (Zhao et al., 2019). Content-based planning approaches were added into neural systems to enhance content relevance (Moryossef et al., 2019; Yao et al., 2019; Hua and Wang, 2019). Hua et al. (2021) extended previous content planning approaches by dividing the content plan into different types

of information; ‘Entities’, ‘Claim’ and ‘Concepts’. An alternative planning approach focuses on where the key phrases should be placed in the generated text in order to improve coherence (Hua and Wang, 2020). The text generator is then conditioned on the provided key phrases and their respective positions in text. Similar to our approach, these methods allow users varying levels of custom control by manual augmentation of the planning module output.

Syntactic or Discourse Control Various works utilised syntactic parse trees with the transformer structure to gain syntactic control or improve interpretability on short-form text generation tasks (Li et al., 2020; Nguyen et al., 2020). With a focus on improving long-form text generation, Ji and Huang (2021) used a Variational Autoencoder (VAE) structure to model RST discourse relations between successive elementary discourse units (spans of text).

We build upon their approach by using a more expressive binary tree formalisation of RST. This formalisation extends the modelling of sequential elementary discourse units by also modelling nuclearity and relationship between non sequential discourse units.

Extending on previous works, Wang et al. (2019b) attempted to couple tree structures and transformers. We instead embed the tree structure of RST into transformers through constrained attention in a separate RST embedding dimension.

3 Rhetorical Structure Theory

Rhetorical Structure Theory (RST) provides a formal structure for interpreting language based on relations between parts of text. Each structure is defined by three sets of features: the binary tree structuring of Discourse Units (EDU)s, the nuclearity between sibling DUs, and the relationship between sibling DUs. An example RST tree using the schema provided by (Mann and Thompson, 1988) as shown in Figure 1.

Binary Tree Structuring of Discourse Units To form a binary discourse tree R from text t , the text must be successively divided into smaller subsets of text. In this manner, node 0 represents the full text, with subsequent nodes representing subtexts, as in Figure 1. The text at each node is called a Discourse Unit, while the nodes at the leaves of the tree are Elementary Discourse Units (EDUs).

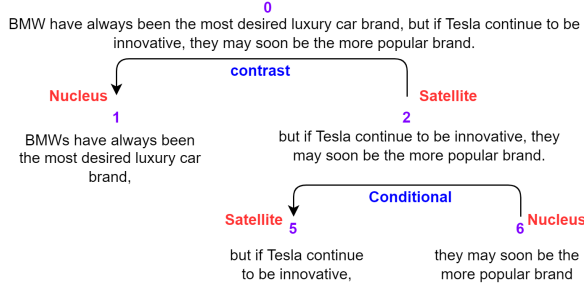


Figure 1: **RST example:** An example text (top) interpreted through RST schema. The text is broken down into a binary tree with zero-indexed **numbered node positions**. **Relations** exist between the two sibling nodes of a parent node. Each pair of sibling nodes, have a coupled **nuclearity** label. Nodes with no children, are Elementary Discourse Units (EDUs).

Sibling Nodes – Relations: A parent node is a non-terminal node which has two child nodes, referred to as siblings. Each parent nodes’ children, has a RST relation $r_i, i \in \mathbb{N}_{<20}$ describing the syntactic purpose of each child node relative to the other. In Figure 1, the RST relations are presented in blue texts.¹

Sibling Nodes – Nuclearity: Associated with each pair of sibling node’s relation is a joint nuclearity labelling, $n_j, j \in \mathbb{N}_{<5}$. This nuclearity explains the role of each sibling node’s discourse unit relative to the other sibling node. Each joint nuclearity labelling between pair of sibling nodes must include at least one Nuclei.²

Formally, we define a binary RST Tree R as a collection of its parent nodes $\{v_l^{r,n}\}$, where $v_l^{r,n}$ is a parent node at position l , with a RST relation r and a Nuclearity n between its two children. For example, $v_2^{\text{Conditional}, \text{SN}}$ represents node 2 in Figure 1. We refer to the group of relations in an RST Tree, R , as R^r . Similarly, the group of Nuclearity values is referred to as R^n .

4 RSTGen: RST-Dependent Long-form Text Generator

We propose a long-form text generation approach, call RSTGen, which provides control over semantics and structure of text, with the aim of improving *coherence* and *cohesion* of the generated text. *Coherence* is concerned with how well constituent

¹Table A1 in the Appendix details the 19 possible RST Relations.

²Table A2 in the Appendix details the 4 possible RST Nuclearity values.

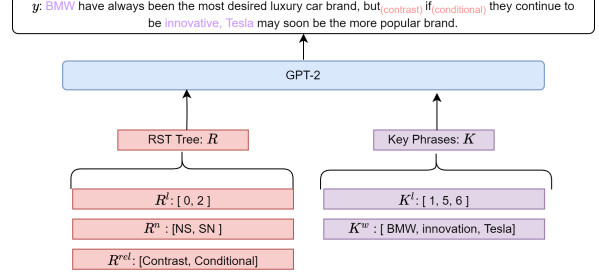


Figure 2: **RSTGen Structure:** Our RST controlled text generation framework can be built upon any pre-trained language model. We input RST tree structure information as R , comprising the Nuclearities R^n , RST Relations R^r and RST node positions R^l , and key phrase information at EDUs as K , comprising the key phrases K^w and the RST node positions of the key phrases, K^l . The discourse structure of the output text y is controlled by R^n and R^l . The syntax of y is controlled by R^n , R^l and K^l . The semantics is controlled by R^r and K^w . Our framework creates a prefix-embedding from the encodings for R and K , after which text is generated as a continuation.

sentences/chunks follow on from the previous sentence/chunk. This can be interpreted as the topic of each subsequent sentence/chunk being relevant to the previous sentence/chunk (Carrell, 1982; Waller, 2015). *Cohesion* describes the way in which text is tied together by linguistic devices such as *Therefore...*, *However...*, *In addition to* (Waller, 2015). This can be interpreted as how smooth the model transitions between the types of sentences. With an RST interpretation, *cohesion* is loosely related to the RST relations between different nodes in the binary tree, while *coherence* is related to the RST relations and key phrases at EDUs.

Task formulation Our input consists of (1) an encoding for a binary RST Tree R , (2) a set of keyphrases information K that are relevant to the text. The RST encoding is formed as sets of three pieces of information for each parent node in the binary RST Tree. These include Nuclearity R^n , RST Relation R^r and RST node position R^l . The keyphrase information contains two pieces of information. These are K^w and K^l , the key phrases and the RST node positions of the key phrases. The model has been trained to work with varying levels of specification for the context information.

Partial Context Provision In practice, we do not expect users to provide the full RST Tree or key phrases since often only a subset of features of the context information may be known

or needed. For example, in Figure 2, the goal is to produce a text contrasting the innovative ability of Microsoft and Tesla. A user only needs to provide partial information, e.g., the keywords, $K^w = [BMW, Tesla, innovation]$ and the RST relation, $R^r \supset ['Contrast']$. Our model will be able to automatically predict an RST tree following a conditional sampling method we have designed in Section 4.3.

While a user will often want to specify the key phrases themselves, we could also employ automated methods to create an expanded set. For example, existing Planning Modules (Hua and Wang, 2020; Hua et al., 2021) can be used to generate a set of keywords based on some prior information.

In what follows, we describe our proposed RST-Gen in more details.

4.1 Tokenisation

While our RST controlled text generation framework can be built upon any pre-trained language model, we use GPT as an example here. The GPT Tokeniser is used to tokenise the target text y and key phrases K^w to a set of word tokens. Two new tokens are added to the tokeniser: '`<rst>`' and '`<kp>`'. The former is prepended to the RST tree R , and the latter is prepended to each key phrase.

4.2 Embedding Layers

Our framework requires three additional embedding layers to facilitate the encoding of the RST position, RST nuclearity and RST relation information presented in R^l , R^n and R^r . These embedding layers are designed to produce vectors that match the size v of hidden vectors in the base model.

We use a RST relation Embedding Matrix $W_r \in \mathbb{R}^{19 \times 768}$ to encode R^r as there are 18 possible RST relations and a pad token, and an RST Nuclearity Embedding matrix $W_n \in \mathbb{R}^{4 \times 768}$ to encode R^n , which consists of 3 possible nuclearity labels and a pad token.

To embed RST node position encodings R^l , we create a novel embedding layer designed to capture the positional relationships of nodes in a binary tree, in a space efficient manner. The intent is to explicitly capture the relationship between a node and its ancestor nodes. Our embedding layer features a non-trainable embedding matrix $W_{pe} \in \mathbb{R}^{\max_rst_node \times tree_depth}$, a trainable feed forward layer $W_{pff} \in \mathbb{R}^{tree_depth \times 768}$ and a Gelu activation layer $f_g(\cdot)$. In our experiments

$\max_rst_node = 4094$ and $\max_tree_depth = 12$.

The i -th column in the non-trainable embedding matrix W_{pe} is a vector representing the position of node i , in terms of a sequence of Lefts (L) and Rights (R) required to travel from the root node 0 to node i . Left is encoded as -0.05 and Right as 0.05 . For example, the vector at node position 5 in W_{pe} in the example RST tree shown in Figure 1 is encoded as $[0.05, -0.05, 0, 0, \dots]$ with the remaining 0s representing padding values up to the length $\max_tree_depth$.

It is important to note that in our RST Tree encoding R , a parent node v_i is labelled with the relation r_i and nuclearity n_i connecting its children. As such our encodings for a R^r , R^l and R^n do not include the leaf nodes with no children, but still represent R . This allows our encodings, for a Binary RST Tree with N nodes to be sequences of length $\lceil N/2 \rceil$.

4.3 RST Predictor

We use an RST Predictor to predict the relation and nuclearity of a child node conditional on the RST relation, nuclearity and position of their parent node. For example we predict the left child node $v_{2l+1}^{r_i, n_j}$, by modelling the following conditional distribution $v_{2l+1}^{r_i, n_j} \sim p(r, n | l_{parent}, r_{parent}, n_{parent}, \mathbb{1}(v_{2l+1} = \text{left child}))$. A full RST Tree can be predicted by iteratively repeating this one-step sampling.

We propose a neural sampler method to estimate the conditional distribution $p(\cdot | \cdot)$. It is trained on the RST Trees observed in the training set of the RST Annotated Dataset introduced in Section 6. Our neural RST Sampler, depicted in Figure 3, uses the BART (Lewis et al., 2019) model. The encoder takes a prompt as input. The decoder is re-initialised and reduced to two layers. The decoder takes four inputs: (1) The parent nodes' relation r ; (2) The parent node's nuclearity n ; (3) The parent node's RST position l ; (4) A vector b indicating whether the target node is a left child node or right child node. This vector b is calculated as the difference between the node position embeddings for the parent node and the target child node encodings.

We use the Embedding Layers structures from our RSTGen model to encode the inputs (1-4) to the Encoder. The BART decoder contains two classification heads, to predict relation and nuclearity.

Two approaches can be used to guide our RST

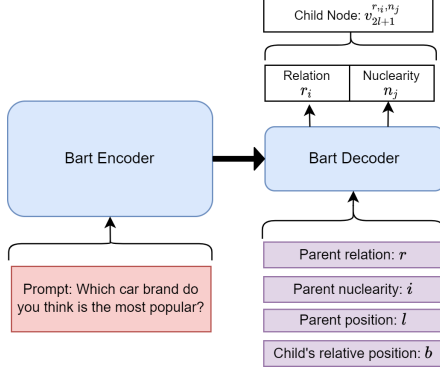


Figure 3: **Neural RST Sampler:** Our Neural RST Sampler can be used to iteratively predict an RST Tree, conditioned on a prompt which is passed to the Encoder. The Decoder takes as input the summation of embeddings for the parent’s nodes relation r , nuclearity n and position l . Further we also pass in b which represents whether the target child node is a left or right child.

predictor to produce more relevant RST Trees. Firstly, the training set can be limited to a specific domain, such as argumentative text, to introduce bias the prediction towards a specific linguistic style. Secondly, during inference the predictive distribution can be augmented, to ensure specific structural properties are achieved. In our experiments, we use this to predict RST Trees with a specific size. We have extended this to other structural features such as specifying RST relation frequency, and position.

4.4 RST-Aware Attention (R.A.A.)

To better encode the RST tree structure in language model training and to improve the coherence of text by reducing the occurrence of hallucination in long-form generated text, we propose a novel attention scheme, called RST-aware attention. In particular, to generate a word token at position i , we create dynamic attention masks m_i allowing the hidden state h_i to focus only on structurally relevant hidden representations h_j , $j \neq i$. The hidden representation h_j is structurally relevant to h_i if h_j ’s associated RST node is an ancestor of the RST node associated to h_i . The RST-aware attention is described in Algorithm 1.

In the above process, we need to first detect the RST node position of the word token to be generated. We do this in a sequential manner with the help of an EDU splitter (Wang et al., 2018), which can detect EDU boundaries in text. At the start of text generation, we set the initial EDU node position as the leftmost leaf node in the RST tree and

Algorithm 1: RST-aware attention

Input : Hidden representations: h_i, h_j , $j \neq i$, and their associated RST nodes v_l, v_m at positions l, m respectively.

Output : Updated hidden representation h'_i .

```

1 for  $j \neq i$  do
2   if  $\text{isParentNode}(v_l, v_m)$  then
3      $m_{i,j} = 1$ 
4   else
5      $m_{i,j} = 0$ 
6 end
7 return  $h'_i \leftarrow \text{MaskedAttention}(h_i, \{h_j\}_{j \neq i}, m_i)$ 

```

then proceed to generate the first token. Afterwards, we use the EDU splitter to detect whether an EDU boundary has occurred. If no boundary is detected, we continue with the generation of the next word token; otherwise, we infer the next EDU node position as the second leftmost leaf node in the RST tree. The above process is repeated until the ‘end of sequence’ token is generated or until the final child EDU node is generated. In practice we use heuristics to avoid the need to perform an EDU boundary detection at each generation step.

5 Open Generation Evaluation

We first train a model using our RST annotated dataset described below. Then we analyse the model’s ability to control semantic, syntactic and text structure.

Subreddit	% of Training Instances
r/CasualConversation	41.7
r/changemyview	19.1
r/DebateReligion	15.8
r/PoliticalDiscussion	9.62
r/relationship_advice	7.94
Other Subreddits	6.84
Total	965,411 samples

Table 1: **Statistics of RST Annotated Dataset:** The majority of our dataset is sourced from 5 subreddits which exhibit a tendency to produce longer and more complex texts.

RST Annotated Dataset We use the ConvoKit API (Chang et al., 2020) to collect a total of over 965k text examples from Reddit to use as the training set. Table 1 provides a detail regarding the division of the dataset between different subreddits. The following subreddits comprise the majority of our dataset, *DebateReligion*, *RelationshipAdvice*, *Politics* and *ChangeMyView*. These subreddits contain many long texts with opinions or persuasive

Input	
Key phrases:	"good mouse alternative" and "none"
Text prompt:	"It would be"
RST structure:	{Relation: r_1 , Nuclearity: SN, Node position: 0}, {Relation: r_2 , Nuclearity: NS, Node position: 2}
Input Relations (r_1, r_2)	Generated Text
Attribution, Explanation	It would be a good mouse alternative, because none of us are going to be able to see.
Background, Evaluation	It would be a good mouse alternative, since none of us have ever seen one.
Background, Condition	It would be a good mouse alternative if none of them were good.
Contrast	It would be a good mouse alternative, but none of them are as good as a good one.

Table 2: **Short Text Semantic control:** Examples of text generated by varying the RST relations. Words highlighted in red in the generated text indicate the corresponding underlined RST input relations.

style of writing. Further, we choose large samples from the subreddit *CasualConversation* to complement the limited range of language present in the former subreddits. Each sample contains a reddit post, its RST tree generated using the best statistical RST parser (Feng and Hirst, 2012) and keyphrases extracted from the post using the PageRank inspired algorithm, TextRank (Mihalcea and Tarau, 2004). The keyphrase extraction process is detailed in Appendix D.

Semantic Control In Table 2 we show the generated short text conditional on various RST relations. For all four examples the same key phrases of "a good mouse alternative" and "none of this" were passed to the model. Furthermore, we provide the first three words "It would be" to the decoder as a text prompt. The text is generated by varying the RST relations, r_1 and r_2 . We can observe that the generated text varies depending on the desired RST relations input to the model.

Structural Discourse Control Here we use a reconstruction style test to evaluate the ability of our model to include the target relations in the generated text at the correct position. To do this, we allow our model to generate text $\hat{y}$ using the RST tree T as the conditioning factor. We then use an RST parser to generate a RST tree $\hat{T}$ from the generated text $\hat{y}$. For each relation r , we calculate a recall score as the proportion of nodes with relation r in T that also occur in $\hat{T}$ at the same position l . The results are shown in Figure 4. We include results for different lengths of text, measured by elementary discourse units.

We observe that our model achieves strong levels of control for the following relations: *Attribution*, *Background*, *Condition*, *Contrast*, *Elaboration*, *Joint*, *Manner-Means*, and *Temporal*. We believe that the weakened performance on *Cause*

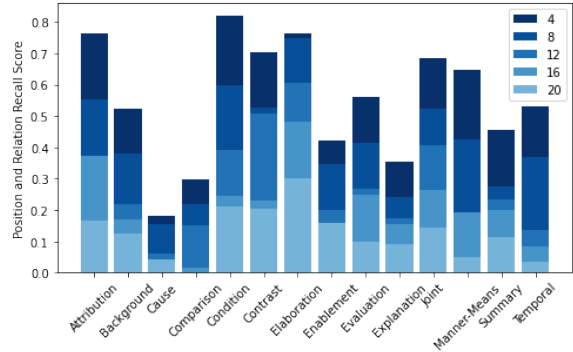


Figure 4: **Structural Discourse Control:** RST relation and position recall scores calculated for each RST relation from the generated text with its lengths measured by the number of Elementary Discourse Units in the range of {4, 8, 12, 16, 20}.

and *Comparison* is due to their respective similarity to *Attribution* and *Contrast*. We omit topic change since our datasets contains texts mostly constrained to a single topic.

Text Length Control By editing the length of the RST encoding, we gain strong control of the length of the generated text. Here we fix the key phrases and vary the length of the RST context (i.e., number of EDUs) from 2 to 14 to demonstrate the increasing length of generated text. Results are depicted in Figure 5. We believe that our method provides a more natural way to control text length when compared to the heuristic methods of fine tuning text generation hyper-parameters such as repetition penalty, length penalty and minimum length.

RST Tree Edit Distance Here we extend the Tree Edit Distance to RST Trees to evaluate how well RSTGen is able generate text that adheres to the RST encoding passed to it. Specifically, we investigate how well this model performs as the

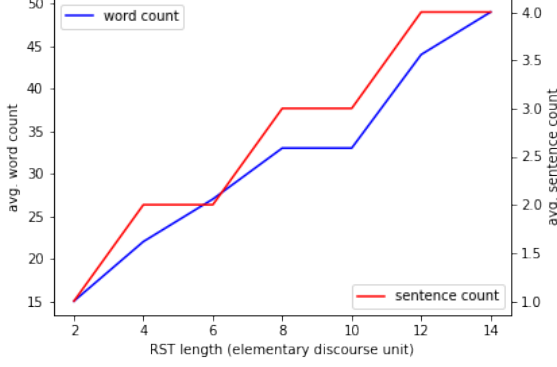


Figure 5: **Text Length Control:** RSTGen exhibits the ability to implicitly control the length of generated text by controlling the size of the RST structure provided as context. In this figure we present the average word count (left y -axis) and average sentence count (right y -axis) against the length of the RST encoding passed to RSTGen.

RST structure becomes increasingly long. For this experiment, we pass an RST Encoding and a set of keywords to our model and generate a text y . We then extract an RST tree from the generated text y using an RST parser. An edit includes the following operations: Changing/Removing the relation or nuclearity label of the node at position l with cost 1; Changing the position of a node to its sibling node with cost 1; and Deleting/Inserting a blank node at position l with cost 3. For a tree of size s we use a normalising factor of $3 \times s$, the cost of creating the tree. Our normalising factor does allow distances over 1. We observe from Figure 6 that generated text is able to adhere to correct RST structure in terms of RST node positions relatively well. However, when considering nuclearity and relation, the inaccuracy of RST structure grows significantly.

6 Evaluation on Argument Generation and Story Generation

We evaluate RSTGen for the tasks of argument generation and story generation. These tasks require our model to output coherent and cohesive text.

6.1 Experimental Setup

We describe the datasets used for fine-tuning our model, baseline models and the methods used to ensure fair comparison. Training detail and hyperparameter setting can be found in Appendix C.

Datasets We use two different datasets for argument generation and story generation, respectively.

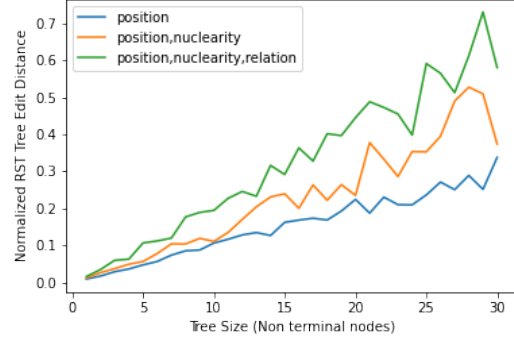


Figure 6: **Normalised RST Tree Edit Distance:** This figure shows the similarity between the RST structure of RSTGen output text and the RST encoding passed as input to RSTGen. We extend Tree Edit Distance to RST Trees to calculate three metrics based on combinations of node position, node nuclearity label and node relation label: (1) Simple Structure: Tree distance including only node positions; (2) Complex Structure: Tree distance including node positions and nuclearity; (3) Complete Structure: Tree distance including node position, nuclearity and relation.

CMV Dataset We use the argument generation dataset first introduced in (Hua et al., 2021). This dataset contains pairs of claims and counter-arguments extracted from titles and responses, respectively, from the Reddit *ChangeMyView* (CMV) forum. This dataset uses posts dated in 2019, which prevents overlap with our training set for the RST Annotated Dataset. The goal of this task is to generate the counter-argument given the initial claim.

Writing Prompts Dataset We use the Story generation dataset created by Fan et al. (2018a) and utilised in (Ji and Huang, 2021). This dataset contains pairs of prompts and stories extracted from the *WritingPrompts* subReddit. The prompt is an introduction to a story that must be extended. We limit each story to be up to 270 tokens.

Baselines For argument generation, we evaluate the following baselines:

CTRL. (Keskar et al., 2019) uses text prompts to control the style of output. The trigger *r/changemyview* style output we prepend the text ‘Opinion Title:’ to the opening statement. This model is approximately 10 times larger than RSTGen and we assume that it achieves performance gains primarily through its larger size.

PAIR (Planning and Iterative Refinement). Hua and Wang (2020) devised a two-step generation process. The first step uses a fine-tuned BERT to

Model	Control Method	Grammar $\uparrow$	Redundancy $\uparrow$	Focus $\uparrow$	Structure and Coherence $\uparrow$
CTRL	Text prompt	0.84	-0.16	-0.12	-0.76
PAIR	Keywords + positions	0.79	0.00	-0.02	-0.78
DYPLOC	Categorised keywords	0.73	-0.10	-0.03	-0.79
RSTGen	RST + PAIR Planner	0.81	-0.03	-0.03	-0.77
RSTGen	RST + DYPLOC Planner	0.83	-0.02	-0.04	-0.75
RSTGen	RST _{predicted} + PAIR Planner	0.75	-0.05	-0.06	-0.88
RSTGen	RST _{predicted} + DYPLOC Planner	0.76	-0.06	-0.05	-0.86
RSTGen w/o R.A.A.	RST + PAIR Planner	0.71	-0.04	-0.08	-0.81
RSTGen w/o R.A.A.	RST + DYPLOC Planner	0.65	-0.06	-0.09	-0.80

Table 3: **Argument Generation Evaluation:** the performance of baseline models and our RST frameworks with different control methods. The shortform R.A.A refers to the RST-aware attention described in Section 4.4.

allocate a position to each pre-defined keyword, in order to create a template for the text to be generated. Then a fine-tuned BART model is used to fill the template.

DYPLOC (Dynamic Planning of Content using Mixed Language Models). Hua et al. (2021) proposed first categorising the keywords into concepts and entities. A BERT-based Planner is used to expand this set of concepts and entities, while another is used to generate the claim. The claim provides a brief summary of the extended answer. Four separate BART Encoders and a BART Decoder are used to convert the expanded set of concepts and entities into an argumentative text.

For both PAIR and DYPLOC, we use their gold standard plan as input to the model, this follows the headline results given in both papers. In pursuit of fair testing, we convert the gold standard plans of PAIR and DYPLOC to two RST encodings. More concretely, for PAIR, we convert its gold standard template, consisting of a set of phrases and their corresponding starting positions in the text to be generated, into a RST encoding. These phrases become the keyphrases for RSTGen. The RST node position for each key phrase can be determined by parsing the true RST structure of the text. For DYPLOC, first we collect the gold standard set of concepts/entities and claims, and filter out any repeated words phrases. Then, the RST node position for each word/phrase can be determined in a similar manner to PAIR.

For Writing Prompts, we compare our proposed approach with the following models:

DiscoDVT (Ji and Huang, 2021). To the best of our knowledge it is the only other work that uses RST to improve the long form text generation ability of a language model. It has three modules: an RST planner, an encoder and a decoder. These mod-

ules are combined to form a VAE structure wherein the RST planner produces a sequence of discrete hidden vectors representing the sequence of RST relations between EDUs in their text. Then at each generation step, sequential hidden vectors are used to guide the text generation process. For fair comparison we do not pass keyphrases to RSTGen.

RSTGen Ablations. We evaluate ablations of RSTGen to validate the two significance features of our proposed RSTGen framework. Firstly, we remove the RST-Aware Attentions (R.A.A.), after which we remove RST Nuclearity, then Relation, and finally node positions.

Metrics We use two sets of metrics. For the Argument Generation experiments, we use the GRUEN (Zhu and Bhat, 2020) set of metrics, which measure four important features of text: Grammaticality, Non-redundancy, Focus and Structure/Coherence. A final metric combines these four features, and has been shown to correlate strongly with human reviewers. For the Story Generation experiments, we use BLEU (Papineni et al., 2002) focusing on the n -gram precision; Distinct- n (Li et al., 2016), which takes the proportion of distinct n -grams relative to all generated n -grams, thus producing a non-reference-based measure for diversity; the GRUEN Structure and Coherence metric (G-SC); and MS-Jaccard (Alihosseini et al., 2019) which uses the Jaccard Index to measure how similar the distribution of n -grams is between two sets of text.

6.2 Evaluation Results

This section presents the performance of RSTGen against baseline models and discusses the performance of RSTGen ablations.

Argument Generation Table 3 shows that our proposed RSTGen using the DYPLOC Planner performs on par with CTRL in terms of *Grammar* despite using only one tenth parameters of CTRL. It outperforms all the baselines in *Structure and Coherence*.

Using automatically predicted RST trees as input, we observe degraded performance of RSTGen results. We believe that this can be attributed to our Markovian RST sampling methodology of sampling a child node dependent on its direct parent node. This may produce RST Trees with unrealistic structures. Nevertheless, it still outperforms CTRL in *Redundancy* and *Focus*, implying that the combination of RST relations, nuclearity and key phrases provide strong content guidance.

RSTGen without the RST-aware attention (R.A.A.) experiences a drop in performance across all metrics, especially *Grammar* and *Focus*. During the generation of longer sequences, using R.A.A. ensures that the same key phrase or RST information does not influence adjacent elementary discourse units, leading to more diversity and less repetition in generated text. We posit that the removal of R.A.A. exacerbates the reduced performance of RSTGen for longer texts, a trait that was exemplified in Figure 6.

Story Generation Table 4 shows that the R.A.A. variants of our model perform on par with the baseline. We observe that the removal of the R.A.A. causes a significant drop in performance, specifically Distinct-4, confirming our findings from the Argument Generation. As the RST relations carries information on the semantics of text, we observe that its removal has a significant effect on the similarity based metrics of BLEU-1, Distinct-4 and MS-Jaccard 3.

Model	BLEU-1 ↑	Distinct-4 ↑	G-S&C ↑	MS-Jaccard 3 ↑
DiscoDVT	24.10	84.66	-0.73	34.76
RSTGen (no keyphrase)	24.01	84.85	-0.71	34.51
- R.A.A.	23.20	82.12	-0.75	33.21
- RST relation	22.28	81.09	-0.77	33.79
- RST nuclearity	22.29	81.12	-0.77	33.78
- RST positions	22.04	80.91	-0.75	33.73

Table 4: **Story Generation Evaluation:** Results for the RSTGen and Baselines on the Writing Prompts story generation dataset. RSTGen performs competitively with DiscoDVT.

7 Conclusion

We present a novel controlled text generation framework, RSTGen, which uses fine-control over a Rhetorical Structure Theory based context as a means to improve the coherence and cohesion of generated text. We also leverage the structural information presented by RST to propose an rst aware attention scheme, ensuring that the model attends to the correct information during long form generation. Through investigation of RSTGen’s open generation text, we showed that our approach can exhibit a high level of interpretable fine-control over syntactic, semantic and structural features of text.

8 Ethics Statement

We acknowledge that our proposed model may be susceptible to learning harmful biases present in the dataset. In and of itself this has the potential to harm minorities, marginalised communities and project stigmas present in society. Further, we recognise that our efforts to improve coherence, cohesion and control might be misused to author offensive or fictitious content. Therefore, we advocate for morally correct and responsible practices in the case of real-world application.

Acknowledgements

This work was funded by the the UK Engineering and Physical Sciences Research Council (grant no. EP/T017112/1, EP/V048597/1). YH is supported by a Turing AI Fellowship funded by the UK Research and Innovation (grant no. EP/V020579/1).

References

- Danial Alihosseini, Ehsan Montahaei, and Mahdieh Soleymani Baghshah. 2019. *Jointly measuring diversity and quality in text generation models. Proceedings of the Workshop on Methods for Optimizing and Evaluating Neural Language Generation*.
- Anusha Balakrishnan, Jinfeng Rao, Kartikeya Upasani, Michael White, and Rajen Subba. 2019. *Constrained decoding for neural NLG from compositional representations in task-oriented dialogue*. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 831–844, Florence, Italy. Association for Computational Linguistics.
- Lynn Carlson, Daniel Marcu, and Mary Ellen Okurowski. 2003. *Building a Discourse-Tagged Corpus in the Framework of Rhetorical Structure Theory*, pages 85–112. Springer Netherlands, Dordrecht.

- Patricia L. Carrell. 1982. [Cohesion is not coherence](#). *TESOL Quarterly*, 16(4):479–488.
- Jonathan P. Chang, Caleb Chiam, Liye Fu, Andrew Z. Wang, Justine Zhang, and Cristian Danescu-Niculescu-Mizil. 2020. [Convokit: A toolkit for the analysis of conversations](#).
- Mingda Chen, Qingming Tang, Sam Wiseman, and Kevin Gimpel. 2019. [A multi-task approach for disentangling syntax and semantics in sentence representations](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2453–2464, Minneapolis, Minnesota. Association for Computational Linguistics.
- Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. 2019. [Plug and play language models: A simple approach to controlled text generation](#).
- Michael Denkowski and Alon Lavie. 2014. [Meteor universal: Language specific translation evaluation for any target language](#). In *Proceedings of the Ninth Workshop on Statistical Machine Translation*, pages 376–380, Baltimore, Maryland, USA. Association for Computational Linguistics.
- Wenyu Du, Zhouhan Lin, Yikang Shen, Timothy J. O’Donnell, Yoshua Bengio, and Yue Zhang. 2020. Exploiting syntactic structure for better language modeling: A syntactic distance approach. In *ACL*.
- Ondřej Dušek and Filip Jurčíček. 2016. Sequence-to-sequence generation for spoken dialogue via deep syntax trees and strings. *arXiv preprint arXiv:1606.05491*.
- William Falcon et al. 2019. Pytorch lightning. *GitHub*. Note: <https://github.com/PyTorchLightning/pytorch-lightning>, 3:6.
- Angela Fan, Mike Lewis, and Yann Dauphin. 2018a. [Hierarchical neural story generation](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 889–898, Melbourne, Australia. Association for Computational Linguistics.
- Angela Fan, Mike Lewis, and Yann Dauphin. 2018b. [Hierarchical neural story generation](#). *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.
- Angela Fan, Mike Lewis, and Yann Dauphin. 2019. Strategies for structuring story generation. *arXiv preprint arXiv:1902.01109*.
- Vanessa Wei Feng and Graeme Hirst. 2012. [Text-level discourse parsing with rich linguistic features](#). In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 60–68, Jeju Island, Korea. Association for Computational Linguistics.
- Tanya Goyal and Greg Durrett. 2020. [Neural syntactic preordering for controlled paraphrase generation](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 238–252, Online. Association for Computational Linguistics.
- Zhiting Hu, Zichao Yang, Xiaodan Liang, Ruslan Salakhutdinov, and Eric P. Xing. 2017. [Toward controlled generation of text](#). In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1587–1596. PMLR.
- Xinyu Hua, Ashwin Sreevatsa, and Lu Wang. 2021. [DYPLOC: Dynamic planning of content using mixed language models for text generation](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6408–6423, Online. Association for Computational Linguistics.
- Xinyu Hua and Lu Wang. 2019. Sentence-level content planning and style specification for neural text generation. *arXiv preprint arXiv:1909.00734*.
- Xinyu Hua and Lu Wang. 2020. [PAIR: Planning and iterative refinement in pre-trained transformers for long text generation](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 781–793, Online. Association for Computational Linguistics.
- Haozhe Ji and Minlie Huang. 2021. [Discodvt: Generating long text with discourse-aware discrete variational transformer](#).
- Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. 2019. [Ctrl: A conditional transformer language model for controllable generation](#).
- Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*.
- Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. 2016. [A diversity-promoting objective function for neural conversation models](#). *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Yinghao Li, Rui Feng, Isaac Rehg, and Chao Zhang. 2020. [Transformer-based neural text generation with syntactic guidance](#).
- William C. Mann and Sandra A. Thompson. 1988. [Rhetorical structure theory: Toward a functional theory of text organization](#). *Text - Interdisciplinary Journal for the Study of Discourse*, 8(3):243–281.

- Rada Mihalcea and Paul Tarau. 2004. [TextRank: Bringing order into text](#). In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 404–411, Barcelona, Spain. Association for Computational Linguistics.
- Amit Moryossef, Yoav Goldberg, and Ido Dagan. 2019. [Step-by-step: Separating planning from realization in neural data-to-text generation](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2267–2277, Minneapolis, Minnesota. Association for Computational Linguistics.
- Xuan-Phi Nguyen, Shafiq Joty, Steven C. H. Hoi, and Richard Socher. 2020. [Tree-structured attention with hierarchical accumulation](#).
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. [Bleu: a method for automatic evaluation of machine translation](#). In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.
- Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners - gpt2.
- Yikang Shen, Shawn Tan, Alessandro Sordoni, and Aaron C. Courville. 2019. Ordered neurons: Integrating tree structures into recurrent neural networks. *ArXiv*, abs/1810.09536.
- Maite Taboada and William C. Mann. 2006. [Applications of rhetorical structure theory](#). *Discourse Studies*, 8(4):567–588.
- Stephen Waller. 2015. [Cohesion is still not coherence, so what is?](#) *English Teaching In China*, (6):31–35.
- Wenlin Wang, Zhe Gan, Hongteng Xu, Ruiyi Zhang, Guoyin Wang, Dinghan Shen, Changyou Chen, and Lawrence Carin. 2019a. Topic-guided variational autoencoders for text generation. *arXiv preprint arXiv:1903.07137*.
- Yaoshian Wang, Hung-Yi Lee, and Yun-Nung Chen. 2019b. [Tree transformer: Integrating tree structures into self-attention](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1061–1070, Hong Kong, China. Association for Computational Linguistics.
- Yizhong Wang, Sujian Li, and Jingfeng Yang. 2018. [Toward fast and accurate neural discourse segmentation](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 962–967, Brussels, Belgium. Association for Computational Linguistics.
- Tsung-Hsien Wen, Milica Gašić, Nikola Mrkšić, Pei-Hao Su, David Vandyke, and Steve Young. 2015. [Semantically conditioned LSTM-based natural language generation for spoken dialogue systems](#). In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1711–1721, Lisbon, Portugal. Association for Computational Linguistics.
- Sam Wiseman, Stuart Shieber, and Alexander Rush. 2017. [Challenges in data-to-document generation](#). In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2253–2263, Copenhagen, Denmark. Association for Computational Linguistics.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2019. [Huggingface’s transformers: State-of-the-art natural language processing](#).
- Lili Yao, Nanyun Peng, Weischedel Ralph, Kevin Knight, Dongyan Zhao, and Rui Yan. 2019. Plan-and-write: Towards better automatic storytelling. In *The Thirty-Third AAAI Conference on Artificial Intelligence (AAAI-19)*.
- Tiancheng Zhao, Kaige Xie, and Maxine Eskenazi. 2019. Rethinking action spaces for reinforcement learning in end-to-end dialog agents with latent variable models. *arXiv preprint arXiv:1902.08858*.
- Wanzheng Zhu and Suma Bhat. 2020. [GRUEN for evaluating linguistic quality of generated text](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 94–108, Online. Association for Computational Linguistics.

A RST Schemas

We list in Table A1 the RST relations and nucleus that our proposed RSTGen framework utilises. We also provide a schema in Table A2 explaining the meaning of Nuclearity labels used by RSTGen.

B Argument and Story Generation Dataset Details

For both the argument generation and story generation experiments our datasets originate from the respective paper’s of models in the baseline study. We do not make any alterations to the datasets, but instead simply use it as provided. We refer the reader to and Hua et al. (2021) for the argument generation dataset (i.e., the CMV dataset) and Fan et al. (2018b) for further details regarding the story generation dataset (i.e., the Writing Prompt dataset).

C Reproducibility

Code The code used to train and evaluate our models can be downloaded from <https://github.com/Rilwan-A/RSTGen>. As well as code, the github will contain links to the RST annotated versions of the CMV dataset and Writing Prompts dataset. Access to the full RST Annotated Reddit dataset can be gained upon request.

Repositories The RSTGen models were extended from pretrained models in Huggingface’s Transformers repository (Wolf et al., 2019). RSTGen is initialised using the GPT2-base model with approximately 124M parameters. The neural RST Predictor was initialised using the BART-base model. We used Pytorch-Lightning (Falcon et al., 2019) for all our training scripts.

Hardware For fine-tuning the RSTGen model on the RST Annotated Datasets, we used 2 GeForce RTX 3090 (24GB). For the Argument Generation Tasks and the Story Generation Task, the RSTGen variants and RST Neural Sampler are fine-tuned using 1 GeForce RTX 3090 (24GB). All training was done using mixed Precision (FP16) to improve memory efficiency.

Fine-tuning For fine-tuning all variations of RSTGen, we used the Adafactor optimiser with the following parameter settings: scale parameter = False, relative step = True, warmup init = True, learning rate = None, weight decay = 0.01. Due to the high computational expense required, we did

not perform extensive hyper-parameter tuning for our RSTGen models.

When fine-tuning on the RST Annotated Dataset, we used an effective batch size of 44 and trained using an Early Stopping rule allowing for at most one epoch to pass with no improvement. The maximum target sequence length is 270 tokens. The maximum RST Tree Size is 36 parent nodes. The maximum Key Phrase sequence size is 64 tokens. These models take approximately 5 epochs to converge which takes approximately 10 hours.

D Keyphrase Extraction using TextRank

Our second conditioning factor is the phrases that are important to the generated text. This importance is determined by the steps listed below:

Noun Chunks and Named Entities Given a text x , the noun chunks and named entities are extracted to form a set of sub-phrases $[x_1^{sp}, x_2^{sp}, \dots, x_N^{sp}] \in x$.

Graph Formation Separately, a graph G is formed from text x by extracting all words w_i that have a Part-Of-Speech tag of either ‘Adjective’, ‘Noun’, ‘Proper Noun’ or ‘VERB’. These words are lemmatised and form nodes $V_i \in G$.

Edge Creation A weighted edge w_{ij} between nodes $V_i, V_j, i \neq j$ has a weight of 1 if the distance between the words corresponding to V_i, V_j is less than some threshold k . In other TextRank implementations factors such as word length, position and frequency can be used to scale w_{ij} .

Node Scoring The TextRank score $S(V_i)$ of a node V_i is initialised to a default value. Then Page Rank’s adapted Eigenvector centrality measure is used to calculate the importance of each node V_i . $S(V_i)$ is iteratively updated using the equation below, until convergence is reached:

$$S_V(V_i) = (1 - d) + \left(d \times \sum_{j \in \mathcal{N}(i)} \frac{w_{ji} \times S(V_j)}{\sum_{k \in \mathcal{N}(j)} w_{jk}} \right)$$

where d is a damping factor and set to 0.85 as in (Mihalcea and Tarau, 2004) and $\mathcal{N}(i)$ are the set of indices of the neighbours of V_i .

Keyphrase Scoring Given the set of node scores $S(V_i)$, the score of a sub-phrase x_n^{sp} , with L words, is computed by summing the scores of the words it contains normalised by its length +1 to favor

Relation	Nuclei	Satellite
Attribution	the effect	the factor which it can be attributed
Background	text whose understanding is being facilitated	text whose understanding is being facilitated
Cause	action/situation	factor which resulted in action/situation's occurrence
Comparison	a comparison between two or more subjects/objects	NA
Condition	action/situation resulting from the occurrence of a conditioning situation	conditioning situation
Contrast	one alternate	the other alternative
Elaboration	basic information	additional information
Enablement	an action/event enabled by a factor	the factor
Evaluation	a situation	an evaluative comment about the situation
Explanation	a statement	the supporting statement to explain the statement
Joint	a list or dis-junction	NA
Manner-Means	the action (being) performed	the manner or means by which the action was performed/achieved
Topic-Comment	a statement such as a question, topic or statement	a paired statement such as an answer / topic-comment or response
Summary	a statement	a restatement, that is shorter
Temporal	a statement with temporal dependence	factor that is depended on
Topic-Change	a shift from this topic A to	a shift to this topic B
Same-Unit	Used to link parts of discourse separated by embedded discourse relation	NA
Textual-Organization	used to link parts of discourse separated by embedded discourse relation	NA
Null	Non-classified	NA

Table A1: **The RST relations Schema:** A schema providing an interpretation for the RST relations our framework utilises. Some of the descriptions are extracted from (Taboada and Mann, 2006) and (Carlson et al., 2003)

Parent Nodes' Nuclearity	Label	Relationship between sibling does
Nuclei-Nuclei	NN	The left sibling's sub-text is equally as important as the right sibling's sub-text.
Nuclei-Sattelite	NS	The left sibling's sub-text is the important part of the parent node's text.
Sattelite-Nuclei	SN	The right sibling's sub-text is the important part of the parent node's text.
Null	Null	Non-classified

Table A2: **A RST Nuclearity Schema:** A schema providing an interpretation of the RST nuclearity our framework uses.

longer n -grams. This is shown in the equation below:

$$S_{kp}(x_n^{sp}) = \frac{\sum_{l=1}^L S_V(V_l)}{L + 1}$$

Keyphrase candidates are then ranked and redundant candidates filtered out. Two candidate phrases are considered redundant if they have the same lemmatised form.

E Argumentation Generation Examples

In this section, we show some example text generated from RSTGen in comparison to baseline models, where text with poor coherence is underlined. We can observe that CTRL generates coherent text but with a shorten length. Both PAIR and DYPLOC exhibit some inconsistency in their generated text. On the contrary, RSTGen generates much longer text with its inconsistency appearing to be more subtle.

CTRL	It is important when you're reading old texts like the Bible to remember that it was made to be analyzed, which the human race has been doing for a couple of hundred years by now. If I were to come up to someone and say "How can you believe in a religion that said all types of animals converged to the Middle East to get on a big boat to be saved?" the person would respond "I have faith. I don't need to question what I have been taught."
PAIR	It is important to remember what reading your texts about your Bible and remember that you've made what you analyzed to human race " but remember about your couple of hundred years ago they were born? <u>I've been taught to think religion, but I've been taught to have types of animals I haven't been in Middle East and I've never been on boat that a saved person to respond to their faith in the US. My question is, do you really?</u>
DYPLOC	I think it's important to read the Bible as a text, because it's important to remember that the Bible was made up and analyzed by the human race for a couple hundred years. The question is, what is it taught to us? How do we know that a person responds to their faith? <u>Religion is the type of animal that converges on a big boat and saves the middle east.</u>
RSTGen	It is a lot more likely to be analyzed by the human race than any other book of text, or even just some couple of important texts that have been written in an attempt to make sense of what we've done for ourselves and our own species. I'm not sure how you can possibly say this without reading all sorts of different perspectives on these topics as opposed to being made up about something like <u>"the world" which has never really been studied at all - it's only ever happened once.</u> The whole point of your argument rests on whether people are actually doing anything wrong with their minds (which isn't always true). It doesn't mean they aren't trying to explain things beyond thinking through them... but rather why there was no reason to believe those questions were asked when someone else said 'why did god exist?' What if one person had already read such answers? If another person hadn't understood science then wouldn't he simply put themselves into question every single-made assumptions?

Table A3: **Argument Generation Example Output:** We highlight parts of the generated text that correspond to poor coherence. In this example we notice at least one inconsistency in the output for PAIR and DYPLOC. However, the inconsistency in the RSTGen appears to be more subtle.